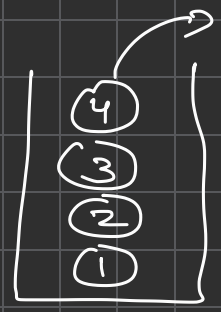
Stacks & Queues

⇒ **Stack:** LIFO

Last In
First Out



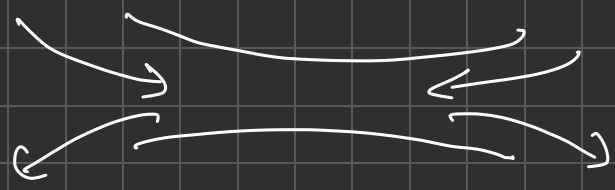
push( ) → O(1)
pop( ) → O(1)

⇒ **Queues:** FIFO

First In
First Out

enqueue( ) – O(1)
dequeue( ) – O(1)

⇒ **Double-ended Queue:**



PROBLEM-1

20. Valid Parentheses

Easy | Topics | Companies | Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

> **Input:** `s = "()"`
>
> **Output:** `true`

Example 2:

> **Input:** `s = "()[]{}"`
>
> **Output:** `true`

Example 3:

> **Input:** `s = "(]"`
>
> **Output:** `false`

Example 4:

> **Input:** `s = "([])"`
>
> **Output:** `true`

s = "( )" , s = "( ) [ ] { }"

n → len string

s = "( ]"

( ) [ ] { } LIFO

Naive Approach

$O(n^2)$

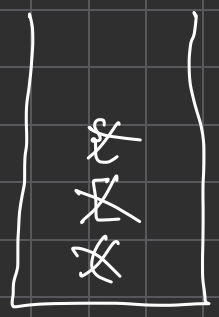
⇓

$O(n)$

Optimal:

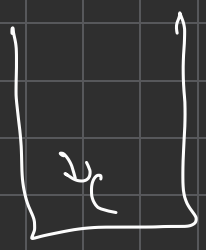
s = "( ) [ ] { }"

valid



{
") " : " ("
"] " : " ["
" } " : " { "
}

s = "( ]"



pseudocode:

```
stack = [] , brackets_map = { }

for (char in string)
    if char in ["(", "[", "{"]:
        stack.append(char)
    else
        if stack and stack[-1] == b_map[char]:
            stack.pop()
        else
            return false
```

return len(stack) == 0

T.C = O(n)
S.C = O(n)

n → len. of string

PROBLEM – 2

Using 2 Queues

225. Implement Stack using Queues

Easy | Topics | Companies

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (`push`, `top`, `pop`, and `empty`).

Implement the `MyStack` class:

- `void push(int x)` Pushes element x to the top of the stack.
- `int pop()` Removes the element on the top of the stack and returns it.
- `int top()` Returns the element on the top of the stack.
- `boolean empty()` Returns `true` if the stack is empty, `false` otherwise.

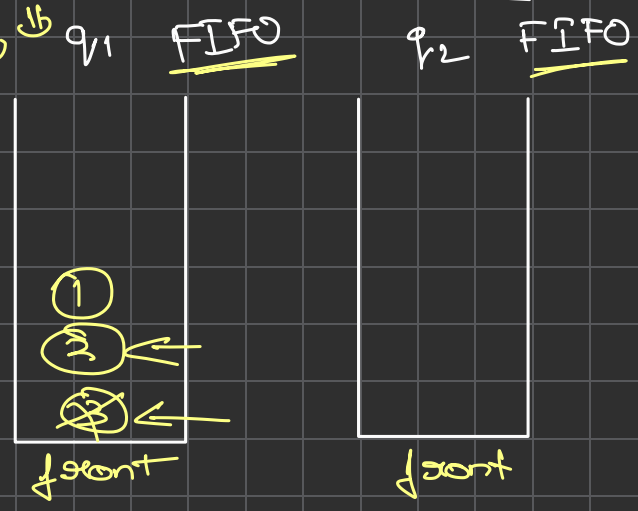
Notes:

- You must use **only** standard operations of a queue, which means that only `push to back`, `peek/pop from front`, `size` and `is empty` operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

```
Input
["MyStack", "push", "push", "top", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 2, 2, false]
```

push(1)
push(2)
push(3)



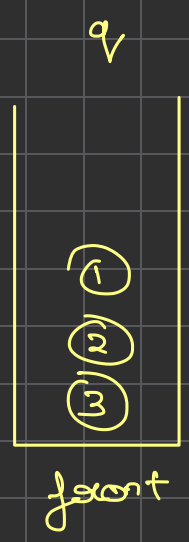
push:

1. enque "element" q_2
2. empty "q_1" and push those to q_2
3. $q_2 \rightarrow q_1$ (deque) (swap)

pop()
top() → 3
empty() →

Using 1 queue

push(1)
push(2)
push(3)



len(q) − 1

PROBLEM - 3

735. Asteroid Collision

Medium | Topics | Companies | Hint

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

```
Input: asteroids = [5,10,-5]
Output: [5,10]
Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.
```

Example 2:

```
Input: asteroids = [8,-8]
Output: []
Explanation: The 8 and -8 collide exploding each other.
```

Example 3:

```
Input: asteroids = [10,2,-5]
Output: [10]
Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.
```

Example 4:

```
Input: asteroids = [3,5,-6,2,-1,4]
Output: [-6,2,4]
Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.
```

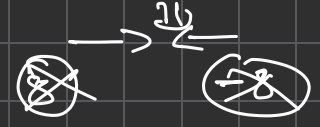
a = [5, 10, -5]

+ve } right

-ve } left

[5, 10]

a = [8, -8]

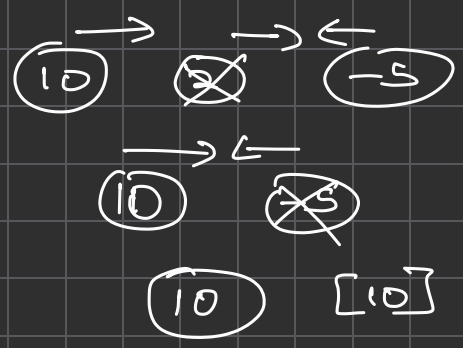


abs(8) = = abs(-8)

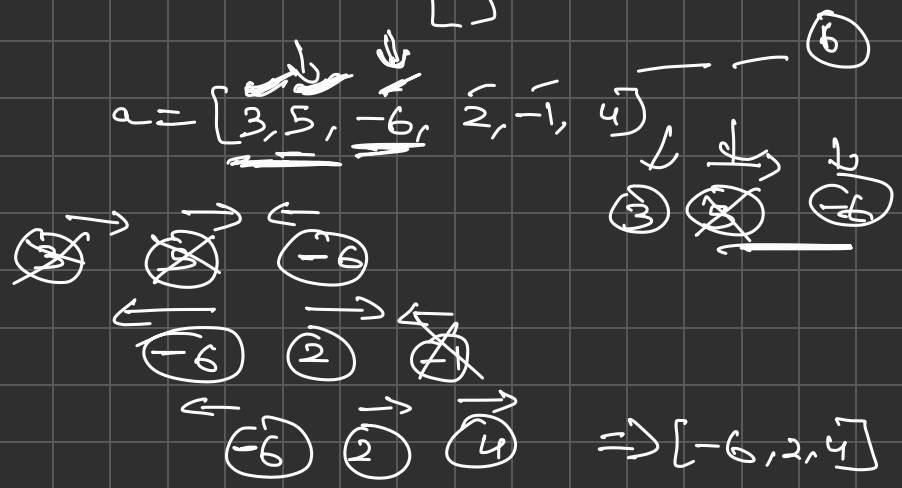
[ ]

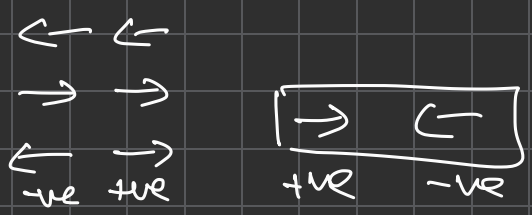
a = [10, 2, -5]

O(n)



a = [3, 5, -6, 2, -1, 4]





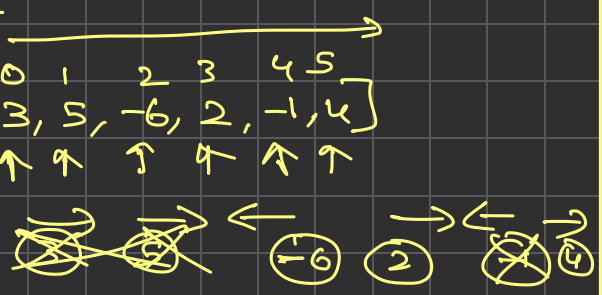
-ve +ve

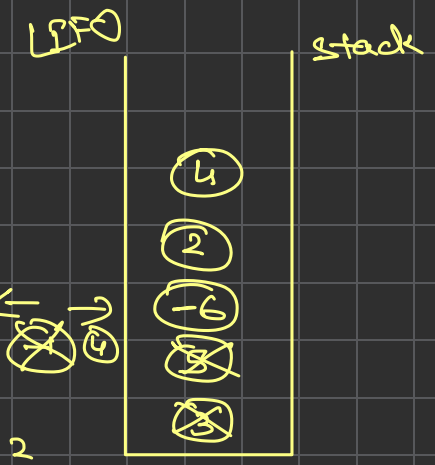
+ve -ve

=> [-6, 2, 4]

Optimal:

$$a = \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ [3, & 5, & -6, & 2, & -1, & 4] \end{array}$$





abs(-1) < 2

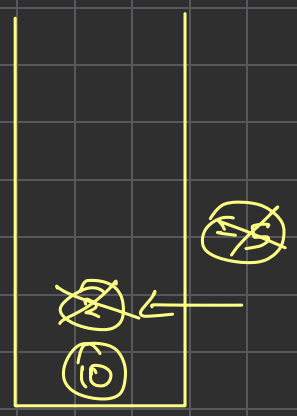
T.C = O(n)

S.C = O(n)

$a = [10, 2, -5]$

visual

[10]



PROBLEM-4

155. Min Stack

Medium | Topics | Companies | Hint

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with `O(1)` time complexity for each function.

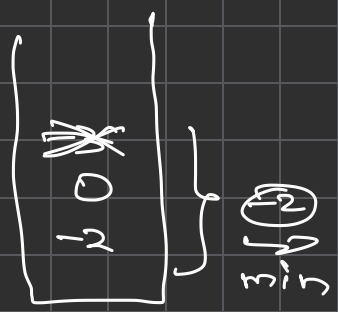
Example 1:

```
Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]

Output
[null,null,null,null,-3,null,0,-2]

Explanation
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top();    // return 0
minStack.getMin(); // return -2
```

Constraints:

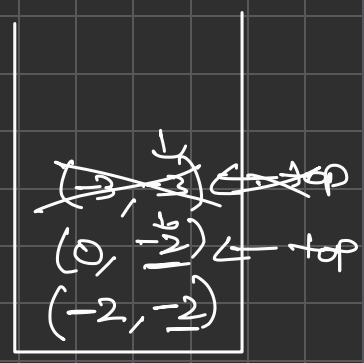
- $-2^{31} <= val <= 2^{31} - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on **non-empty** stacks.
- At most $3 * 10^4$ calls will be made to `push`, `pop`, `top`, and `getMin`.

push(-2)
push(0)
push(-3)
getMin() -> -3
pop()
top()
getMin()



(ele, min-so-far)

push(-2)
push(0)
push(-3)
getMin() -> -3
pop()
top() -> 0
getMin() -> -2 O(1)



PROBLEM - 5

739. Daily Temperatures

Medium | Topics | Companies | Hint

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `i^th` *day to get a warmer temperature*. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

```
Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]
```

Example 2:

```
Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]
```

Example 3:

```
Input: temperatures = [30,60,90]
Output: [1,1,0]
```

Constraints:

- `1 <= temperatures.length <= 10^5`
- `30 <= temperatures[i] <= 100`

0 1 2 3 4 5 6 7

t = [73, 74, 75, 71, 69, 72, 76, 73]

ans = [0, 0, 0, 0, 0, 0, 0, 0]

1 1 4 2 1 1 0 0

Naive approach:

```
ans = [0]*n
for i: 0 -> n-1:
    for j: i+1 -> n:
        if (t[j] > t[i])
            ans[i] = j - i
            break
```

T.C = $O(n^2)$

S.C = $O(1)$

Optimal:

0 1 2 3 4 5 6 7

t = [73, 74, 75, 71, 69, 72, 76, 73]

ans = [0, 0, 0, 0, 0, 0, 0, 0]

1 1 4 2 1 1 0 0

TC = $O(n)$

SC = $O(n)$

$O(n)$

LIFO

(73, 7)

(76, 6)

~~(72, 5)~~

~~(75, 2)~~

(temp, index)